```
################################################################################
# Global Carbon Emission Inequality, Moving Beyond Conventional Analyses
# Author: Atiq Kainan Ahmed. Email: atiqka@yahoo.com
################################################################################

import matplotlib
matplotlib.use("Agg")

import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import ineqpy.inequality as ineq

# ====================================================
# SETTINGS & CREDITS
# ====================================================

MACRO_FILE = "/Users/atiq/Atiq_R-test/macro6_data_17Jan2025.csv"
OUTPUT_DIR = "/Users/atiq/Pyvs_2026/"

AUTHOR_CREDIT = "Developed by: Atiq Kainan Ahmed | Email: atiqka@yahoo.com"

# ====================================================
# LOAD & MERGE DATA
# ====================================================

print("Loading macro dataset...")
macro = pd.read_csv(MACRO_FILE)
macro["countryname"] = macro["countryname"].astype(str).str.strip()
macro["inc_grp"] = macro["inc_grp"].astype(str).str.strip()
macro["pop"] = pd.to_numeric(macro["pop"], errors="coerce")

print("Downloading OWID CO2 dataset...")
url = "https://raw.githubusercontent.com/owid/co2-data/master/owid-co2-data.csv"
co2 = pd.read_csv(url)

latest_year = int(co2["year"].max())
print(f"Using latest available year: {latest_year}")

co2 = co2[co2["year"] == latest_year].dropna(subset=["co2_per_capita"])
co2.rename(columns={"country": "countryname"}, inplace=True)
co2["countryname"] = co2["countryname"].astype(str).str.strip()

# Merge & Clean
df = macro.merge(co2[["countryname", "co2_per_capita"]], on="countryname", how="inner")
df = df[["countryname", "inc_grp", "pop", "co2_per_capita"]].dropna()
print(f"Total matched countries: {len(df)}")

DATA_SOURCE_TEXT = f"Data Source: Our World in Data (OWID) CO2 Dataset ({latest_year}) & Macro Dataset"


# ====================================================
# PLOT 1: Global Carbon Lorenz Curve (Ranked Globally)
# ====================================================
print("Generating Plot 1: Global Carbon Lorenz Curve...")

df_sorted = df.sort_values("co2_per_capita").reset_index(drop=True)
```

```
total_pop = df_sorted["pop"].sum()
total_co2 = (df_sorted["co2_per_capita"] * df_sorted["pop"]).sum()

df_sorted["cum_pop"] = df_sorted["pop"].cumsum() / total_pop
df_sorted["cum_co2"] = (df_sorted["co2_per_capita"] * df_sorted["pop"]).cumsum() / total_co2

cum_pop_1 = np.insert(df_sorted["cum_pop"].to_numpy(), 0, 0.0)
cum_co2_1 = np.insert(df_sorted["cum_co2"].to_numpy(), 0, 0.0)

fig, ax = plt.subplots(figsize=(9, 7))
ax.plot([0, 1], [0, 1], linestyle="--", color="gray", linewidth=1.5, label="Line of Equality")
ax.plot(cum_pop_1, cum_co2_1, color="navy", linewidth=2.5, label="Global Lorenz Curve")
ax.fill_between(cum_pop_1, cum_co2_1, cum_pop_1, alpha=0.3, color="lightsteelblue")

ax.set_title("Framing 1: Global Carbon Inequality (Countries Ranked by Per-Capita Emissions)\n", fontsize=13,
fontweight="bold")
ax.set_xlabel("Cumulative Share of Global Population", fontsize=11)
ax.set_ylabel("Cumulative Share of Global CO₂ Emissions", fontsize=11)
ax.grid(True, linestyle=":", alpha=0.7)
ax.legend(loc="upper left", fontsize=10)

# Footers
fig.text(0.12, 0.015, DATA_SOURCE_TEXT, fontsize=8, style='italic', color='dimgray')
fig.text(0.98, 0.015, AUTHOR_CREDIT, fontsize=8, style='italic', color='dimgray', ha='right')
plt.tight_layout(rect=[0, 0.05, 1, 1])

path_1 = OUTPUT_DIR + "lorenz_framing1_global.png"
plt.savefig(path_1, dpi=600, bbox_inches="tight")
plt.close()
print(f"Saved: {path_1}")


# ======================================================
# PLOT 2: Emission-Tier Faceting (Instead of Income)
# ======================================================
print("Generating Plot 2: Emission-Tier Faceting...")

def assign_emission_tier(val):
    if val < 3.0:
        return "Low Emitters (< 3t)"
    elif val < 8.0:
        return "Moderate Emitters (3–8t)"
    elif val < 15.0:
        return "High Emitters (8–15t)"
    else:
        return "Very High Emitters (> 15t)"

df["emission_tier"] = df["co2_per_capita"].apply(assign_emission_tier)
tier_order = ["Low Emitters (< 3t)", "Moderate Emitters (3–8t)", "High Emitters (8–15t)", "Very High Emitters (> 15t)"]

fig, axes = plt.subplots(2, 2, figsize=(12, 10))
axes = axes.flatten()

for i, tier in enumerate(tier_order):
    subset = df[df["emission_tier"] == tier].copy()

    if len(subset) < 3:
```

```
        axes[i].text(0.5, 0.5, "Insufficient data in tier", ha="center", va="center")
        axes[i].set_title(tier)
        continue

    values = subset["co2_per_capita"].to_numpy()
    weights = subset["pop"].to_numpy()

    lorenz_df = ineq.lorenz(values, weights=weights)
    gini_val = ineq.gini(values, weights=weights)

    axes[i].plot(lorenz_df.iloc[:, 0], lorenz_df.iloc[:, 1], color="darkgreen", linewidth=2, label="Lorenz Curve")
    axes[i].plot([0, 1], [0, 1], linestyle="--", color="red", linewidth=1.5, label="Line of Equality")
    axes[i].fill_between(lorenz_df.iloc[:, 0], lorenz_df.iloc[:, 1], lorenz_df.iloc[:, 0], alpha=0.3, color="darkseagreen")

    axes[i].set_title(f"{tier}\nInternal Gini = {gini_val:.3f}", fontsize=11, fontweight="bold")
    axes[i].set_xlabel("Cumulative Share of Population (Tier)", fontsize=10)
    axes[i].set_ylabel("Cumulative Share of CO₂ Emissions", fontsize=10)
    axes[i].grid(True, linestyle=":", alpha=0.7)
    axes[i].legend(fontsize=9)

fig.suptitle("Framing 2: Internal Inequality within Per-Capita Emission Tiers", fontsize=14, fontweight="bold", y=0.98)
fig.text(0.1, 0.01, DATA_SOURCE_TEXT, fontsize=8, style='italic', color='dimgray')
fig.text(0.98, 0.01, AUTHOR_CREDIT, fontsize=8, style='italic', color='dimgray', ha='right')
plt.tight_layout(rect=[0, 0.04, 1, 0.95])

path_2 = OUTPUT_DIR + "lorenz_framing2_emission_tiers.png"
plt.savefig(path_2, dpi=600, bbox_inches="tight")
plt.close()
print(f"Saved: {path_2}")


# ======================================================
# PLOT 3: Comparative Lorenz Overlay (High vs Low Emitters)
# ======================================================
print("Generating Plot 3: Comparative Lorenz Overlay...")

median_emitter = df["co2_per_capita"].median()
df_high = df[df["co2_per_capita"] >= median_emitter].copy()
df_low = df[df["co2_per_capita"] < median_emitter].copy()

def compute_group_lorenz(sub_df):
    sub_sorted = sub_df.sort_values("co2_per_capita").reset_index(drop=True)
    t_pop = sub_sorted["pop"].sum()
    t_co2 = (sub_sorted["co2_per_capita"] * sub_sorted["pop"]).sum()
    c_pop = np.insert((sub_sorted["pop"].cumsum() / t_pop).to_numpy(), 0, 0.0)
    c_co2 = np.insert(((sub_sorted["co2_per_capita"] * sub_sorted["pop"]).cumsum() / t_co2).to_numpy(), 0, 0.0)
    return c_pop, c_co2

p_high, c_high = compute_group_lorenz(df_high)
p_low, c_low = compute_group_lorenz(df_low)

fig, ax = plt.subplots(figsize=(9, 7))
ax.plot([0, 1], [0, 1], linestyle="--", color="gray", linewidth=1.5, label="Line of Equality")
ax.plot(p_high, c_high, color="firebrick", linewidth=2.5, label=f"Higher-Emitting Bloc (>= Median {median_emitter:.1f}t)")
ax.plot(p_low, c_low, color="dodgerblue", linewidth=2.5, label=f"Lower-Emitting Bloc (< Median {median_emitter:.1f}t)")

ax.set_title("Framing 3: Comparative Lorenz Overlay (High vs. Low Emitter Blocs)\n", fontsize=13, fontweight="bold")
```

```
ax.set_xlabel("Cumulative Share of Group Population", fontsize=11)
ax.set_ylabel("Cumulative Share of Group CO₂ Emissions", fontsize=11)
ax.grid(True, linestyle=":", alpha=0.7)
ax.legend(loc="upper left", fontsize=10)

fig.text(0.12, 0.015, DATA_SOURCE_TEXT, fontsize=8, style='italic', color='dimgray')
fig.text(0.98, 0.015, AUTHOR_CREDIT, fontsize=8, style='italic', color='dimgray', ha='right')
plt.tight_layout(rect=[0, 0.05, 1, 1])

path_3 = OUTPUT_DIR + "lorenz_framing3_comparative_overlay.png"
plt.savefig(path_3, dpi=600, bbox_inches="tight")
plt.close()
print(f"Saved: {path_3}")


# ======================================================
# PLOT 4: Combined View (All Three Framings in One Plot)
# ======================================================
print("Generating Plot 4: Combined All-in-One Framing...")

fig, ax = plt.subplots(figsize=(10, 8))

ax.plot([0, 1], [0, 1], linestyle=":", color="gray", linewidth=1.5, label="Line of Equality")

tier_colors = {
    "Low Emitters (< 3t)": "#2ca02c",
    "Moderate Emitters (3–8t)": "#ff7f0e",
    "High Emitters (8–15t)": "#d62728",
    "Very High Emitters (> 15t)": "#4b0082"
}

df_sorted["emission_tier"] = df_sorted["co2_per_capita"].apply(assign_emission_tier)

prev_x, prev_y = 0.0, 0.0
for tier in tier_order:
    tier_sub = df_sorted[df_sorted["emission_tier"] == tier]
    if tier_sub.empty:
        continue

    seg_x = np.insert(tier_sub["cum_pop"].to_numpy(), 0, prev_x)
    seg_y = np.insert(tier_sub["cum_co2"].to_numpy(), 0, prev_y)

    ax.plot(
        seg_x, seg_y,
        color=tier_colors[tier],
        linewidth=3.2,
        label=f"Global Curve: {tier}"
    )
    prev_x, prev_y = seg_x[-1], seg_y[-1]

ax.plot(
    p_high, c_high,
    linestyle="--",
    color="firebrick",
    linewidth=2.0,
    alpha=0.85,
    label=f"Bloc Overlay: Higher-Emitting (>= {median_emitter:.1f}t)"
```

```
)
ax.plot(
    p_low, c_low,
    linestyle="--",
    color="dodgerblue",
    linewidth=2.0,
    alpha=0.85,
    label=f"Bloc Overlay: Lower-Emitting (< {median_emitter:.1f}t)"
)

ax.set_title(
    "Combined Framing: Global Carbon Lorenz Curve, Emission Tiers & Bloc Overlays\n",
    fontsize=13,
    fontweight="bold"
)
ax.set_xlabel("Cumulative Share of Population (Global or Group)", fontsize=11)
ax.set_ylabel("Cumulative Share of CO₂ Emissions (Global or Group)", fontsize=11)
ax.grid(True, linestyle=":", alpha=0.6)
ax.legend(loc="upper left", fontsize=9, frameon=True, facecolor="white", framealpha=0.9)

fig.text(0.12, 0.015, DATA_SOURCE_TEXT, fontsize=8, style="italic", color="dimgray")
fig.text(0.98, 0.015, AUTHOR_CREDIT, fontsize=8, style="italic", color="dimgray", ha="right")
plt.tight_layout(rect=[0, 0.05, 1, 1])

path_4 = OUTPUT_DIR + "lorenz_framing4_combined.png"
plt.savefig(path_4, dpi=600, bbox_inches="tight")
plt.close()
print(f"Saved: {path_4}")

print("\nSaved all figures!")

# End of script #
```